

Tabla de símbolos

```
public class Simbolo {  
    private String nombre;  
    private Tipo tipo;  
  
    public Simbolo(String nombre, Tipo tipo) {  
        this.nombre = nombre;  
        this.tipo = tipo;  
    }  
  
    public Simbolo(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public Tipo getTipo() {  
        return tipo;  
    }  
  
    public String toString() {  
        return nombre + " : " + tipo;  
    }  
}
```

Cuando se desarrolla la tabla de símbolos utilizando un lenguaje de programación orientada a objetos, cada categoría de símbolo se diseña como una clase y se organiza una jerarquía de clases cuya superclase es **Símbolo** y las subclases podrían ser: Variable, Función, Tipo de dato, etc.

```
public interface Tipo {  
    String getNombre();  
}
```

Para representar de manera general un tipo de dato, se utilizará una interfaz.

```
public class TipoIncorporado extends Simbolo implements Tipo {  
    public TipoIncorporado (String nombre) {  
        super(nombre);  
    }  
  
    public String toString() {  
        return getNombre();  
    }  
}
```

Para representar los tipos de datos incorporados, es decir, los que ya vienen definidos con el lenguaje de programación, podemos implementar una clase como esta.

```
public class Variable extends Simbolo {  
    public Variable(String nombre, Tipo tipo) {  
        super(nombre, tipo);  
    }  
}
```

La categoría de símbolo mas simple son las variables y podemos representarlas con esta clase.

```

public class TablaSimbolos {
    private ArrayList<Simbolo> simbolos = new ArrayList();

    public void definir(Simbolo simbolo) throws SemanticException {
        for (Simbolo s: simbolos)
            if (simbolo.getNombre().equals(s.getNombre()))
                throw new SemanticException(" El símbolo " +
                                            s.getNombre() +
                                            " ya fue declarado");

        simbolos.add(simbolo);
    }

    public Simbolo resolver(String nombre) throws SemanticException {
        for (Simbolo s: simbolos)
            if (s.getNombre().equals(nombre))
                return s;

        throw new SemanticException(" El símbolo " +
                                    nombre +
                                    " no ha sido declarado");
    }

    public ArrayList<Simbolo> getSimbolos() {
        return simbolos;
    }
}

```

El lenguaje de pseudocódigo que se utiliza, solo contiene símbolos tipo variable y el tipo de datos de todas las variables es real (números de punto flotante), por lo que solo se requiere una tabla de símbolos simple

```
public class PseudoParser {  
    private ArrayList<Token> tokens;  
    private int indiceToken = 0;  
    private SyntaxException ex;  
  
    private TablaSimbolos ts;  
    private TipoIncorporado real;  
  
    public PseudoParser(TablaSimbolos ts) {  
        this.ts = ts;  
    }  
}
```

La tabla de símbolos debe incorporarse al analizador sintáctico elaborado anteriormente.

Antes de iniciar el análisis se define en la tabla de símbolos el tipo de datos real.

```
public void analizar(PseudoLexer lexer) throws SyntaxException {  
    tokens = lexer.getTokens();  
  
    real = new TipoIncorporado("real");  
    ts.definir(real);  
  
    if (Programa()) {  
        if (indiceToken == tokens.size()) {  
            System.out.println();  
            System.out.println("La sintaxis del programa es correcta");  
            return;  
        }  
    }  
  
    if (ex == null)  
        ex = new SyntaxException("Error de sintaxis");  
  
    throw ex;  
}
```

```
// <Programa> -> inicio-programa <Enunciados> fin-programa
private boolean Programa() throws SemanticException {
    if (match(TipoToken.INICIOPROGRAMA))
        if (Declaracion())
            if (Enunciados())
                if (match(TipoToken.FINPROGRAMA))
                    return true;

    return false;
}
```



```
// <Declaracion> -> variables : <Variables>
// <Variables> -> , VARIABLE <Variables> | VARIABLE
private boolean Declaracion() throws SemanticException {
    if (match(TipoToken.VARIABLES))
        if (match(TipoToken.DOSPUNTOS))
            if (match(TipoToken.VARIABLE)) {
                ts.definir(new Variable(tokens.get(indiceToken-1).getNombre(), real));
                System.out.println("Definida");

                while (match(TipoToken.COMA)) {
                    if (!match(TipoToken.VARIABLE))
                        return false;

                    ts.definir(new Variable(tokens.get(indiceToken-1).getNombre(), real));
                    System.out.println("Definida");
                }

                return true;
            }

    return false;
}
```

```
// <Asignacion> -> VARIABLE = <Expresion>
private boolean Asignacion() throws SemanticException {
    int indiceAux = indiceToken;
    Variable v;

    if (match(TipoToken.VARIABLE)) {
        v = (Variable) ts.resolver(tokens.get(indiceToken - 1).getNombre());
        System.out.println("Resuelto");

        if (match(TipoToken.IGUAL))
            if (Expresion())
                return true;
    }

    indiceToken = indiceAux;
    return false;
}
```

```

public class PruebaTablaSimbolos {
    public static void main(String[] arg) throws LexicalException, SyntaxException {
        String entrada = leerPrograma("../ejemplo.alg");
        PseudoLexer lexer = new PseudoLexer();
        lexer.analizar(entrada);

        System.out.println("*** Análisis léxico ***\n");

        for (Token t: lexer.getTokens()) {
            System.out.println(t);
        }

        System.out.println("\n*** Análisis sintáctico ***\n");

        TablaSimbolos ts = new TablaSimbolos();
        PseudoParser parser = new PseudoParser(ts);
        parser.analizar(lexer);

        System.out.println("\n*** Tabla de símbolos ***\n");

        for (Simbolo s: ts.getSimbolos())
            System.out.println(s);
    }
}

```

Para probar el funcionamiento del **PseudoParser** con tabla de símbolos, se extiende la clase de prueba que se utilizó anteriormente para probar el análisis léxico y sintáctico.

```
private static String leerPrograma(String nombre) {  
    String entrada = "";  
  
    try {  
        FileReader reader = new FileReader(nombre);  
        int character;  
  
        while ((character = reader.read()) != -1)  
            entrada += (char) character;  
  
        reader.close();  
        return entrada;  
    } catch (IOException e) {  
        return "";  
    }  
}
```

```

inicio-programa
    variables: numeroDeElementos, promedio, i, elemento

    leer numeroDeElementos
    promedio = 0

    repite (i, 1, numeroDeElementos)
        leer elemento
        promedio = promedio + elemento
    fin-repite

    promedio = promedio / numeroDeElementos
    escribir "El promedio es ", promedio
fin-programa

```

```

*** Análisis sintáctico ***

INICIOPROGRAMA: inicio-programa
LEER: leer
VARIABLE: numeroDeElementos
Definida
VARIABLE: promedio
Definida
IGUAL: =
NUMERO: 0
NUMERO: 0
VARIABLE: i
Definida
IGUAL: =
NUMERO: 0
NUMERO: 0
MIENTRAS: mientras
PARENTESISIZQ: (
Resuelta
OPRELACIONAL: <
VARIABLE: numeroDeElementos
Resuelta
PARENTESISISDR: )
LEER: leer
VARIABLE: elemento
Definida
VARIABLE: promedio
Resuelta
IGUAL: =
VARIABLE: promedio
Resuelta
OPARITMETICO: +
VARIABLE: elemento
Resuelta
VARIABLE: i
Resuelta
IGUAL: =
VARIABLE: i
Resuelta
OPARITMETICO: +
NUMERO: 1
FINMIENTRAS: fin-mientras
VARIABLE: promedio
Resuelta
IGUAL: =
VARIABLE: promedio
Resuelta
OPARITMETICO: /
VARIABLE: numeroDeElementos
Resuelta
ESCRIBIR: escribir
CADENA: "El promedio es "
COMA: ,
VARIABLE: promedio
Resuelta
FINPROGRAMA: fin-programa

La sintaxis del programa es correcta

```

*** Tabla de símbolos ***

```

real
numeroDeElementos : real
promedio : real
i : real
elemento : real

```

Ejercicio

- En los ejercicios anteriores se extendió el lenguaje de pseudocódigo para permitir la declaración de las variables utilizadas en un programa y se modificaron las clases **PseudoLexer** y **PseudoParser** para detectar las nuevas unidades léxicas en la declaración y revisar que la sintaxis de una declaración fuera correcta.
- ¿Cómo se podría modificar el **PseudoParser** para registrar las variables declaradas en la tabla de símbolos?
- ¿Cómo se podría modificar el **PseudoParser** para detectar cuando se hace referencia a una variable que no fue declarada ?
- Modifica la clase **PseudoParser** y las clases relacionadas que se requieran, utilizando la tabla de símbolos, para permitir la detección de variables no declaradas y resolver las variables que si fueron registradas en la tabla de símbolos.